

Custos / Clew 프로젝트 종합 보고서

멀티에이전트 AI 낭비 탐지 오피저버빌리티

작성일 2026-06-12

목차

목차	2
1. 한눈에 보기 (30초 요약)	3
2. 우리는 무엇을 만드는가 (정체성)	5
2.1 핵심 가설 (한 문단)	5
2.2 이름의 유래 (Anthropic-Palantir식 작명)	5
3. 여기까지의 여정	6
3.1 v1 'EmergenceTrace' — 가설을 죽이고, 그걸 성공이라 불렀다	6
3.2 피벗 — '실패 예측'에서 '낭비 탐지'로	7
3.3 Stage 1 — 탐지 로직보다 '데이터 기반과 검증 장치'를 먼저	8
3.4 Stage 2 — 캐스케이드 탐지기를 만들고 합격선을 통과하다	8
4. 핵심 기술 개념, 쉽게 이해하기	12
4.1 왜 '캐스케이드(2단 구조)'인가	12
4.2 4가지 낭비 패턴	12
4.3 '개발용 셋 / 평가용 셋' 분리가 왜 중요한가	13
5. 현재 위치 — 검증된 것 vs 아직 아닌 것	14
6. 시장과 경쟁	15
6.1 시장 규모	15
6.2 경쟁 지형과 우리의 갭	15
7. 관련 논문 종합	17
8. 우리의 진짜 해자 — '자기기만 방지' 규율	18
9. 앞으로의 방향	19
9.1 기술 — 다음 마일스톤	19
9.2 제품 로드맵 (연구↔제품 플라이휠)	19
9.3 시장 진입(GTM) — 준비된 자산	19
10. 대표로서 꼭 기억할 것	20

1. 한눈에 보기 (30 초 요약)

우리는 **Custos**라는 회사에서 **Clew**라는 제품을 만들고 있습니다. 여러 AI 에이전트가 협업할 때 똑같은 일을 헛되이 반복하거나 의미 없이 주고받으며 토큰(=돈)을 태우는 낭비 구간을 자동으로 찾아내는 도구입니다. 개발은 전부 Claude Code CLI로 진행합니다.

지금까지의 성과를 한 줄로 말하면 — “합성 데이터로 만든 탐지기가, 사전에 약속한 합격선을 단 한 번의 평가에서 통과했다(F1 0.857, 오탐 0)” 입니다. 다만 아직 실제 프로덕션 트레이스에 서의 검증과 실제 고객은 0 이라는 점이 가장 중요한 현재 한계입니다.

쉽게 말하면 ‘AI 여러 명이 회의하다가 같은 말을 빙빙 돌려서 시간(=비용)을 낭비하는 순간’을 잡아내는 감시 장치를 만들고 있습니다. 실험실(합성 데이터)에서는 잘 작동하는 걸 정직하게 증명했고, 이제 ‘실제 현장’에서도 통하는지 확인하는 단계 직전입니다.	
핵심 항목	현재 상태
회사 / 제품	Custos(회사) · Clew(제품)
하는 일	멀티에이전트 트레이스에서 중복 루프·낭비 핸드오프 탐지
기술 구조	구조→의미 2단 캐스케이드 탐지기
최신 결과	단발 평가 GO: F1 0.857, 정밀도 1.0, 오탐률 0
검증된 것	시장(문제 존재) · 메커니즘(외부 논문) · 합성 데이터 탐지
아직 아닌 것	실제 트레이스 검증 · 실측 토큰 절감액 · 고객/수요
다음 단계	실제 멀티에이전트 트레이스에 탐지기 적용

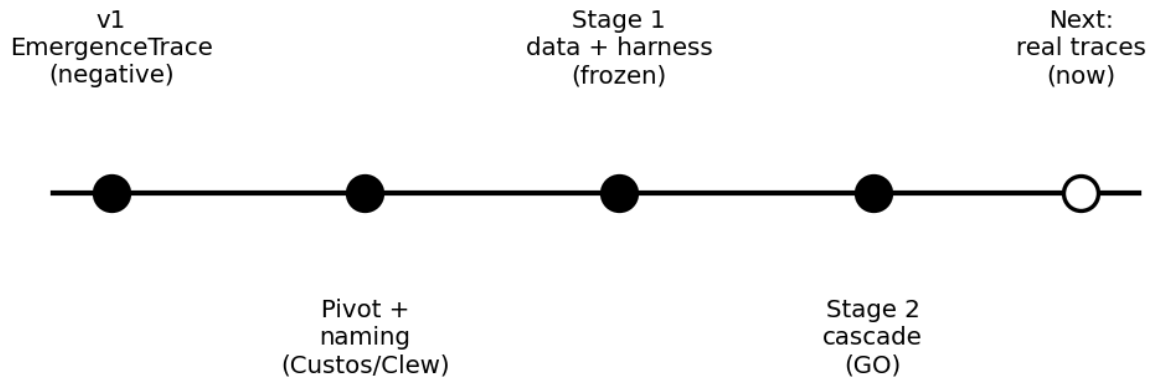


그림 1. 프로젝트 여정 — v1 실패에서 Stage 2 통과까지, 지금은 실제 트레이스 단계 직전

2. 우리는 무엇을 만드는가 (정체성)

2.1 핵심 가설 (한 문단)

멀티에이전트 AI의 실패는 대개 '모델이 멍청해서'가 아니라 **에이전트들 사이에서 벌어지는 문제** — 중복 작업, 헛도는 루프, 낭비되는 핸드오프 — 이고, 이걸 곧 돈(토큰 낭비)입니다. 기존 관측 도구는 대부분 '에이전트 한 명'을 보는 데 맞춰져 있어 이 '사이의 층'을 못 봅니다. Clew는 바로 그 낭비를 **보이게 만들어(→ 결국 막아)** 비용과 신뢰성을 지킵니다.

쉽게 말하면

여러 직원(에이전트)이 한 프로젝트를 같이 할 때, 서로 소통이 꼬여서 '이미 한 일을 또 하고', '쓸데없이 서류를 주고받는' 일이 생깁니다. 그게 다 인건비(토큰 비용)죠. 기존 도구는 직원 '한 명'의 성과만 보지만, 우리는 직원들 '사이'의 낭비를 봅니다.

2.2 이름의 유래 (Anthropic·Palantir 식 작명)

이름은 Anthropic(미션을 고전 어근에 인코딩) + Palantir(기능을 신화에 인코딩)의 작명 원리를 따랐습니다.

이름	어원 / 뜻	답은 의미
Custos (회사)	라틴어 '수호자·파수꾼' (custody의 어근)	에이전트를 지켜보는 자 — 미션
Clew (제품)	아리아드네가 테세우스에게 준 실타래, 영어 'clue'의 어원	미로(에이전트 호출의 미로) 속 길을 추적하는 실 — 기능

정직한 한계 메모: 도메인(clew.ai 등)과 상표(USPTO/KIPRIS)는 **확정 전 직접 확인** 필요합니다. 흔한 단어라 도메인은 변형(useclew, clewhq 등)이 필요할 수 있습니다.

3. 여기까지의 여정

프로젝트는 네 번의 큰 국면을 지나왔습니다. 각 국면이 곧 프로젝트 채팅 한 건씩에 대응합니다.

3.1 v1 'EmergenceTrace' — 가설을 죽이고, 그걸 성공이라 불렀다

처음 아이디어는 멀티에이전트 실패를 **조기 예측**하는 것이었습니다. 두 가지 신호를 썼습니다: (1) Loop Pressure(에이전트 간 메시지 그래프의 구조적 사이클), (2) Information Gain Decay(임베딩 신규도 붕괴). 가설은 '이 둘이 실패를 더 잘 예측한다'였습니다.

결과를 보기 전에 **합격선을 미리 못박았습니다(AUC ≥ 0.80)**. UC 버클리의 MAST-Data(실제 트레이스 1,600여 건, 7개 프레임워크)로 돌린 결과는 — **AUC ≈ 0.455 . 사실상 동전 던지기였습니다.**

게다가 더 나빴습니다. 신호가 '실패 여부'가 아니라 사실상 '**트레이스 길이**'를 재고 있었습니다 (상관 $r \approx 0.86$). 길이를 보정하니 AUC는 0.42로, 방향까지 뒤집혔습니다(성공한 트레이스가 오히려 정보 붕괴가 더 컸음, $p \approx 0.01$).

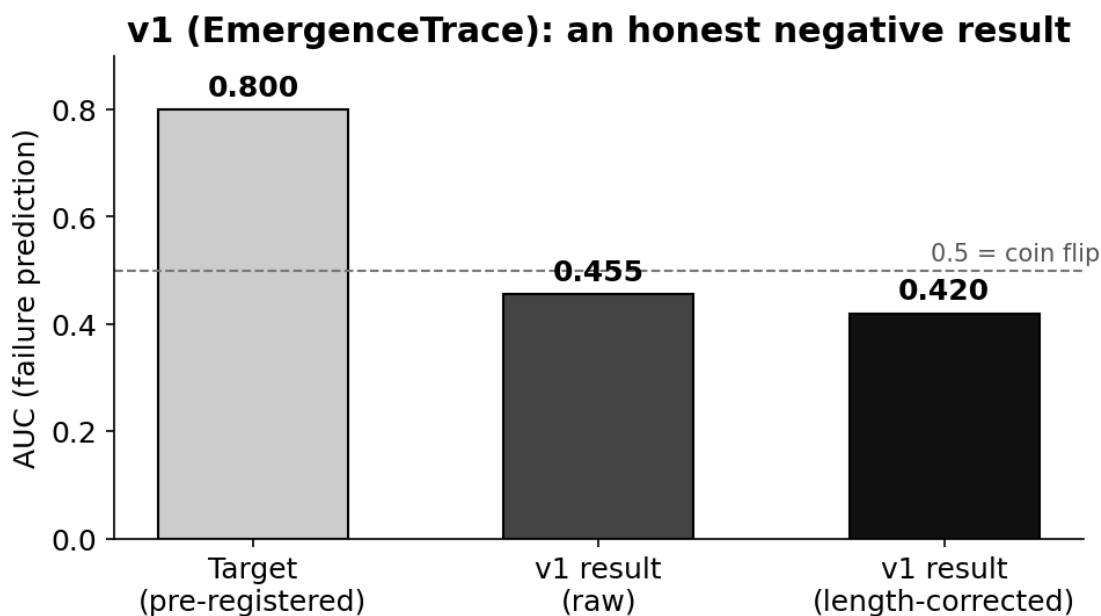


그림 2. v1 검증 — 사전 약속(0.80)에 한참 못 미친 정직한 음성(negative) 결과

쉽게 말하면

'이 신호로 실패를 예측할 수 있다'고 믿었는데, 실험해보니 그 신호는 사실 '글이 긴지 짧은지'만 재고 있었습니다. 실패 예측엔 쓸모가 없었던 거죠. 중요한 건 — 이걸 '몇 주 만에, 매몰비용 없이, 정직하게' 알아냈다는 점입니다. '내 아이디어를 죽일 수 있도록 허락된 실험'을 설계한 것 자체가 성공입니다.

3.2 피벗 — '실패 예측'에서 '낭비 탐지'로

v1의 직관(구조+의미를 같이 봐야 한다)은 사실 옳았습니다. 틀린 건 **구현 방식**이었죠. 이걸 확인해 준 외부 연구가 IBM Research의 논문(arXiv:2511.10650)입니다 — **구조로 후보를 찾고 → 의미 유사도로 중복을 확인하는 '캐스케이드'가 F1 0.72** 를 냈습니다. 구조만 쓰면 0.08, 의미만 쓰면 0.28에 그쳤고요.

그래서 사업의 '뿔개(wedge)'를 바꿨습니다: **'실패를 예측한다'(모호·검증 실패) → '중복 루프.핸드오프로 태우는 토큰 낭비를 탐지·절감한다'(측정 가능한 비용 스토리)**. 이 피벗이 지금 Clew의 정체성입니다.

쉽게 말하면

'미래를 점치는 일(실패 예측)'은 어려웠습니다. 대신 '지금 눈앞에서 벌어지는 명백한 낭비(같은 일 반복)를 잡는 일'로 방향을 틀었습니다. 이건 '얼마를 아꼈는지' 숫자로 보여줄 수 있어서 사업적으로 훨씬 설득력이 있습니다.

3.3 Stage 1 — 탐지 로직보다 '데이터 기반과 검증 장치'를 먼저

탐지 코드를 한 줄 쓰기 전에, 속이지 않는 검증 환경부터 만들었습니다. 이게 v1의 교훈을 구조에 박은 부분입니다.

- **표준 트레이스 모델:** OpenInference/OTel 기반의 span 트리를 pydantic 으로 정의, LangGraph 어댑터 구축.
- **검증 데이터셋:** 80 개 트레이스(정상 40 / 낭비 40), 4 가지 낭비 패턴(repeat_node, regen_handoff, pingpong_abandon, requery_known).
- **합격/중단 기준 동결:** CRITERIA_FROZEN.md 에 $GO(F1 \geq 0.80$ 그리고 오탐률 ≤ 0.10), $KILL(F1 < 0.60$ 또는 오탐률 $> 0.25)$ 을 결과 보기 전에 박음.
- **3 중 누수 차단:** 탐지 코드가 정답 라벨을 절대 읽지 못하도록 정적분석·런타임·모듈 검사 3 겹으로 막음.

Stage 1은 git 커밋 **0fa25e0 (태그 stage1-freeze)**, 테스트 92개 통과로 동결했습니다.

쉽게 말하면

시험 문제를 풀기 전에 '커닝이 절대 불가능한 시험장'과 '채점 기준'을 먼저 만들어 못 박았습니다. 그래야 나중에 좋은 점수가 나와도 '진짜 실력'임을 믿을 수 있으니까요.

3.4 Stage 2 — 캐스케이드 탐지기를 만들고 합격선을 통과하다

핵심 탐지기를 2단 구조로 만들었습니다: 구조 단계가 '반복 후보'를 찾고, 의미 단계가 '진짜 중복인지' 확인합니다.

Structure-then-semantics cascade

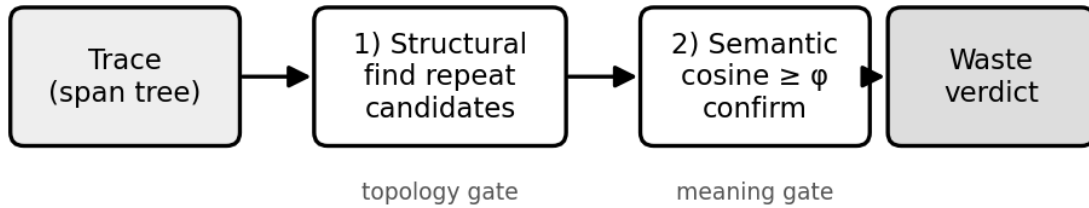


그림 3. 구조→의미 캐스케이드 — 토폴로지로 후보를 거르고, 의미 유사도로 낭비를 확정

엄격한 규율 두 가지를 지켰습니다. 첫째, **파라미터는 오직 '개발용 셋(seed=7)'에서만 정하고, '평가용 셋(seed=42)'은 동결 후 단 한 번만 건드린다**. 둘째, 처음 만든 보정 코드가 '개발 라벨에 과적합'되는 걸 잡아내, 라벨 의존 F1 격자탐색 대신 **분포 분리 기반(P10/P90 간격, Cohen's d) 보정**으로 교체했습니다.

중간에 '통과한 수정이 사실 거짓말이었던' 순간도 잡았습니다 — 구조 후보 생성기가 토폴로지만 보다가, 스키마는 같지만 값이 다른 조회 쌍을 낭비로 오인할 뻔했죠. **tool span에 '입력 텍스트 동일성' 게이트를 추가**해 막았고, 테스트 146개가 통과했습니다.

출시 직전 실사에서 한 가지 구멍을 정직하게 발견했습니다: 4개 패턴 중 **regen_handoff만 0/10(미탐지)**. 진단 결과 '구조적으로 후보 경로가 없는' 문제로 판명돼, 이 패턴은 **v1 로드맵으로 명시적 디스코프(범위 제외)**했습니다 — 단, 게이트에서 빠져 점수를 구제하지 않고 '미탐 10건'을 집계에 불리하게 그대로 반영했습니다.

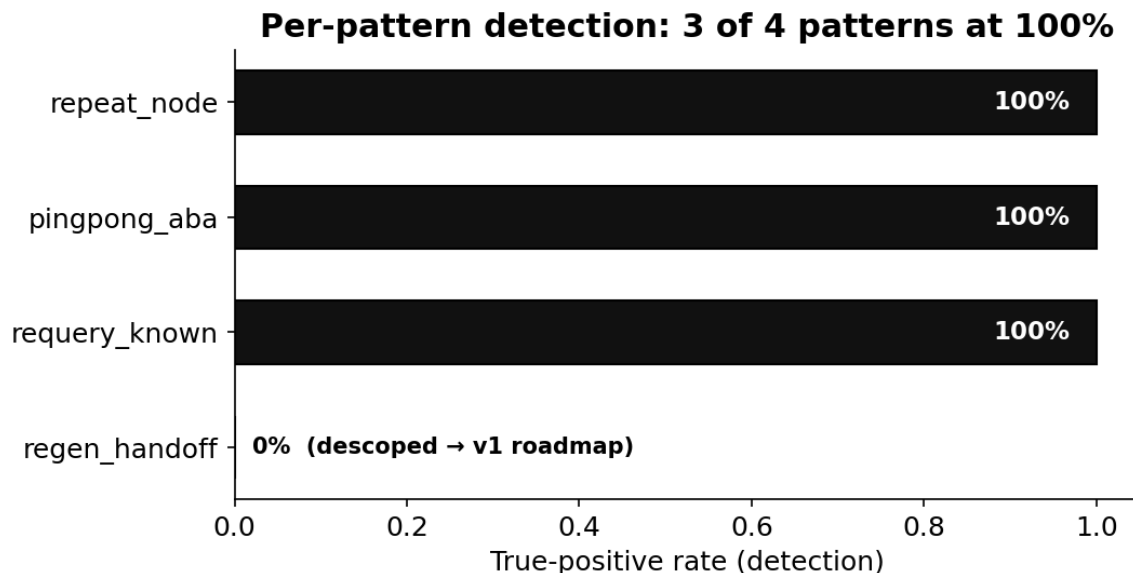


그림 4. 패턴별 탐지율 — 4개 중 3개는 100%, regen_handoff는 0%(로드맵으로 이관)

그리고 단 한 번 허용된 평가(seed=42)를 실행했습니다. 결과는 **GO — F1 0.857, 정밀도 1.0, 인스코프 탐지율 1.0, 트레이스 오탐률 0**. regen 미탐을 불리하게 물리기도 합격선 0.80을 넘겼습니다. 파라미터는 $\phi=0.514345$, $N=2$ 로 동결(커밋 1fadadc)했습니다.

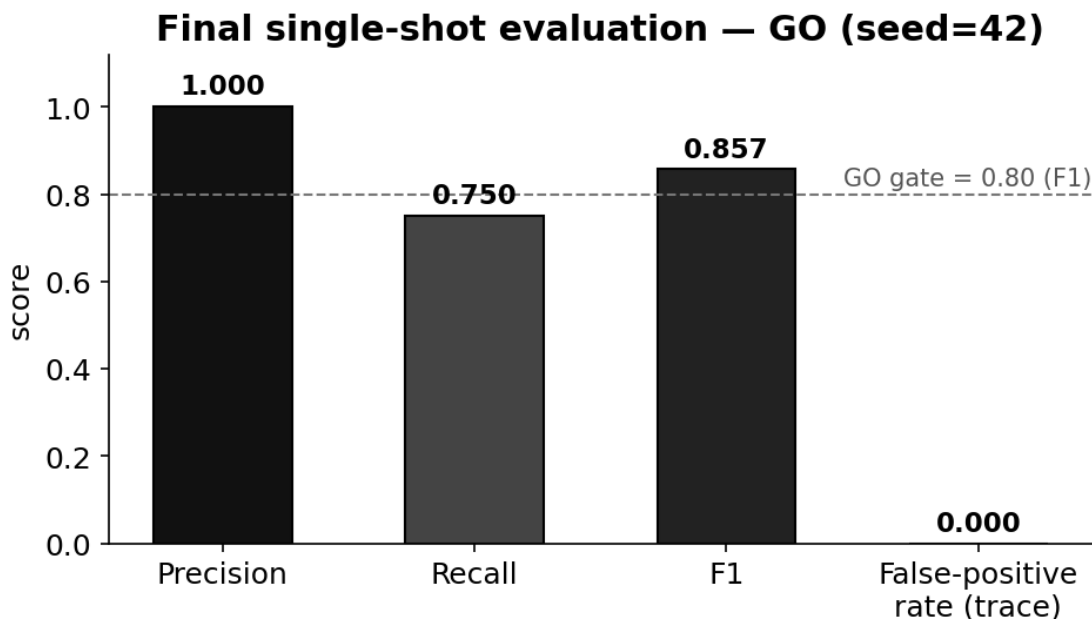


그림 5. 최종 단발 평가 — 사전 약속한 GO 게이트(0.80)를 정직하게 통과

추가로, 실제 OpenInference 계측 트레이스 한 건으로 변환 테스트를 돌려 ‘**실제 트레이스에서 오탐 0이 처음으로 살아남는**’ 증거도 확보했습니다(가짜 탐지 → 소거 확인).

쉽게 말하면

‘낭비 잡는 기계’를 만들어, 미리 정한 합격 기준을 단 한 번의 공식 시험에서 통과했습니다 (F1 0.857). 못 잡는 패턴 하나는 숨기지 않고 ‘아직 못 잡음’으로 솔직히 표시한 채 점수를 받았습니다. 이 정직함이 결과를 더 믿을 수 있게 만듭니다.

4. 핵심 기술 개념, 쉽게 이해하기

4.1 왜 '캐스케이드(2 단 구조)'인가

구조만 보거나 의미만 봐서는 둘 다 형편없습니다. 둘을 '순서대로' 결합할 때 비로소 쓸 만해집니다.

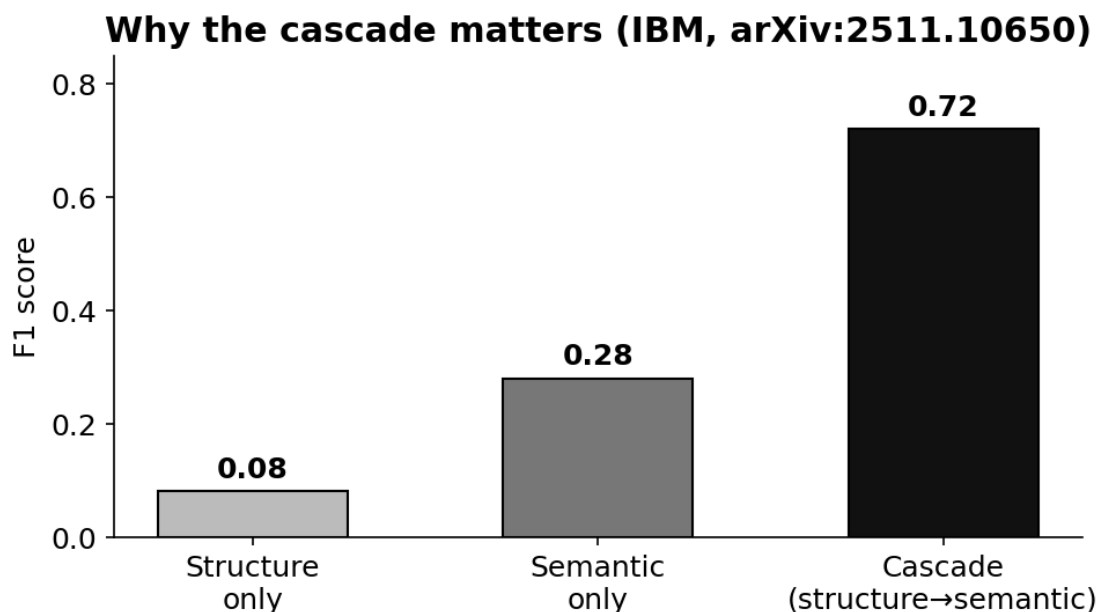


그림 6. 구조 단독 0.08, 의미 단독 0.28, 캐스케이드 0.72 (IBM, arXiv:2511.10650)

쉽게 말하면

'같은 자리를 두 번 지나갔다(구조)'만 보면 정상적인 반복까지 낭비로 오해합니다. '말이 비슷하다(의미)'만 보면 우연히 닮은 것까지 잡습니다. 먼저 구조로 '수상한 후보'만 추리고, 그 다음 의미로 '진짜 중복인지' 확인하면 정확해집니다. 1차 거름망 → 2차 정밀검사 순서입니다.

4.2 4 가지 낭비 패턴

패턴	무엇인가	현재 탐지
repeat_node	같은 노드(에이전트)가 같은 일을 반복	100%
pingpong_aba	$A \rightarrow B \rightarrow A$ 처럼 핑퐁하며 헛도는 왕복	100%
requery_known	이미 아는 정보를 또 조회	100%

패턴	무엇인가	현재 탐지
regen_handoff	핸드오프 과정에서 내용 재생성 낭비	0% (로드맵 이관)

4.3 '개발용 셋 / 평가용 셋' 분리가 왜 중요한가

파라미터를 '평가용 데이터'를 보면서 맞추면, 시험 문제를 미리 보고 공부하는 것과 같아 점수가 '가짜로' 좋아집니다. 그래서 파라미터는 개발용(seed=7)에서만 정하고, 평가용(seed=42)은 동결 후 딱 한 번만 채점에 씁니다. 기록 로그까지 '단 한 번' 규율에 포함됩니다.

5. 현재 위치 — 검증된 것 vs 아직 아닌 것

대표로서 가장 또렷이 들고 있어야 할 표입니다. 과장하지 않는 것이 곧 우리의 신뢰 자산입니다.

구분	상태	설명
시장 (문제 존재)	검증됨	공개 데이터로 확인 — 멀티에이전트 낭비·비용은 실재하는 1순위 페인
메커니즘 (이론)	검증됨(외부)	캐스케이드는 IBM 논문에서 F1 0.72 — 단, 우리 성과가 아니라 외부 연구
탐지 (합성 데이터)	검증됨(우리)	단발 평가 GO, F1 0.857, 오탐 0 — 단, 전부 합성 트레이스
탐지 (실제 트레이스)	아직 아님	실제 프로덕션 트레이스 검증 미완 — 여기서 깨질 가능성 존재
실측 절감액	아직 아님	‘토큰 얼마 절감’ 실측 0건 — 비용 스토리의 핵심 증거가 비어 있음
고객 / 수요	아직 아님	사용자 0명, 지불 의사 미검증

대표가 외워둘 한 문장

“합성 검증에서 오탐 0으로 낭비를 잡는다”까지는 사실이지만, “프로덕션 토큰을 실제로 절약한다”는 아직 말할 수 없습니다. 이 경계를 넘는 표현(예: 4패턴 전부 탐지, 실제 절감 보장)은 모든 대외 커뮤니케이션에서 금지선입니다.

6. 시장과 경쟁

6.1 시장 규모

LLM 옵저버빌리티 시장은 2026년 약 \$2.69B → 2030년 \$9.26B (연 36% 성장) 으로 추정됩니다. 기업 AI의 72%가 이미 멀티에이전트(2024년 23% → 급등)이고, 관측은 스택에서 만족도가 가장 낮은(<33%) 영역이라 교체 기회가 큼니다.

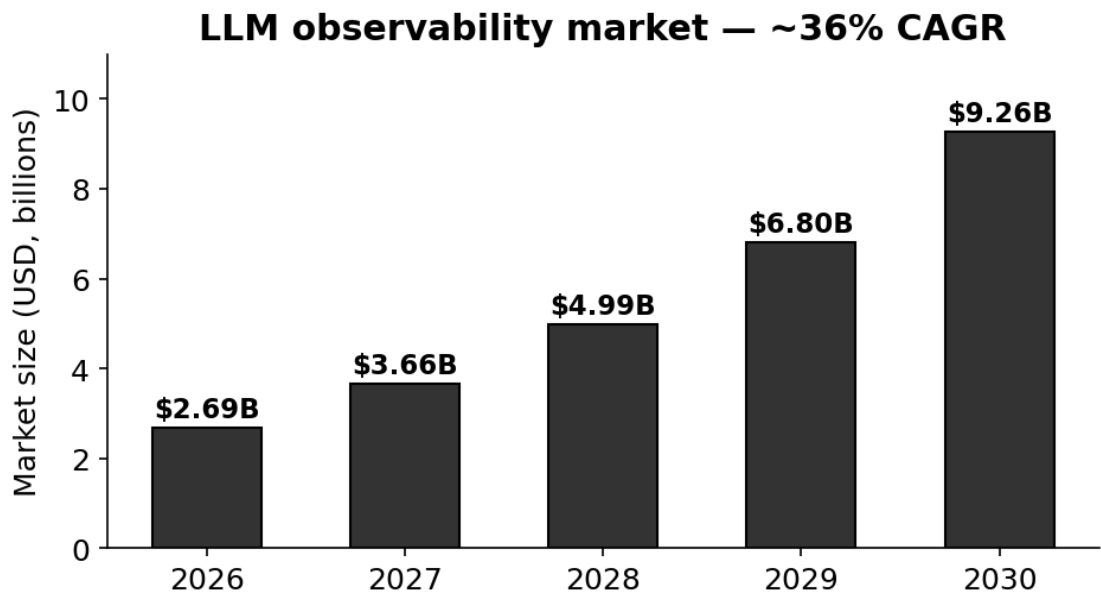


그림 7. LLM 옵저버빌리티 시장 성장 추정 (출처: researchandmarkets / Confident AI)

6.2 경쟁 지형과 우리의 갭

붐비고 통합되는 시장입니다. 범용 영역은 피하고, '멀티에이전트 특화·프레임워크 무관'이라는 좁은 갭을 노립니다.

경쟁자 / 동향	포지션
Langfuse	오픈소스 리더 — 2026년 1월 ClickHouse가 약 \$400M에 인수
LangSmith / Laminar	에이전트 디버깅·실시간 트레이싱
Arize / Phoenix, Datadog	범용 LLM·ML 옵저버빌리티 (레드오션)
우리(Clew)의 갭	에이전트 '사이'의 중복·낭비 특화, 프레임워크 무관, 비용 절감 스토리

경쟁자 / 동향	포지션
쉽게 말하면 큰 회사들이 'AI 전반을 감시하는 도구'를 이미 팝니다. 우리는 그들과 정면 승부하지 않고, '여러 AI가 협업할 때의 낭비'라는 좁고 깊은 틈새를 파고듭니다. 경쟁사를 꺾아내리지 않고 '다른 층, 다른 규율'로 포지셔닝합니다.	

7. 관련 논문 종합

프로젝트의 기술·시장 판단을 떠받치는 핵심 레퍼런스입니다. 인용할 때 '외부 결과'와 '우리 결과'를 절대 섞지 않습니다.

논문 / 출처	핵심 내용	우리에게 의미
IBM Research — George et al. arXiv:2511.10650 'Unsupervised Cycle Detection in Agentic Applications'	구조→의미 캐스케이드 F1 0.72 (정밀도 0.62 / 재현율 0.86). 구조 단독 0.08, 의미 단독 0.28. 1,575개 LangGraph trajectory에서 측정(저자 자체 벤치마크, 채택 후 공개 예정)	캐스케이드 구조의 핵심 근거. v1 직관은 옳고 구현이 틀렸음을 입증. 'IBM 데이터·공개셋이 아님'을 정확히 명기할 것
MAST — Cemri et al. (UC Berkeley) arXiv:2503.13657 HF: mcemri/MAST-Data	멀티에이전트 실패 분류체계. 7개 프레임워크 주석 트레이스 약 1,642건. 실패의 ~44% 시스템 설계 / ~32% inter-agent 정렬 / ~24% 검증	v1 검증 데이터셋. '실패는 에이전트 사이에서 생긴다'는 thesis의 근거. (건수는 1,642 또는 '1,600+'로 표기 — 과거 '1,242'는 재확인 필요)
CaRT — arXiv:2510.08517	조기 종료·교정 관련 신호 연구	교정 신호 설계 참고
Early-Exit — arXiv:2505.17616	에이전트 조기 종료 메커니즘	종료 판단·낭비 차단 참고
Aegean — arXiv:2512.20184	에이전트 안정성/모니터링 관련	향후 단계(차단·거버넌스) 참고

대표가 주의할 점

IBM의 F1 0.72는 '남의 실험 결과'입니다. 우리 성과(F1 0.857, 합성)와 절대 섞어 말하면 안 됩니다. 또 MAST 트레이스 건수는 과거 문서에 '1,242'로 잘못 적힌 적이 있어, 대외 공개 글에서는 원논문 기준(1,642 또는 '1,600+')으로 맞춰야 합니다.

8. 우리의 진짜 해자 — ‘자기기만 방지’ 규율

이 프로젝트의 가장 큰 차별점은 기술이 아니라 **방법론적 정직함**입니다. AI(Claude Code)로 빠르게 개발할수록 ‘스스로를 속이기’ 쉬워지는데, 그걸 구조로 막았습니다.

- **사전 등록:** 결과를 보기 전에 성공/실패 기준을 못 박는다(CRITERIA_FROZEN.md).
- **사후 조정 금지:** 결과를 본 뒤 파라미터를 바꿔 점수를 구제하지 않는다.
- **단발 평가:** 평가용 셋은 동결 후 단 한 번만, 기록 로그까지 포함해 건드린다.
- **누수 차단:** 탐지 코드가 정답 라벨을 읽지 못하게 3 겹으로 막는다.
- **빨간불이면 멈춘다:** regen 0/10 처럼 불리한 결과도 숨기지 않고 집계에 그대로 반영한다.
- **인간이 최종 게이트:** git 동결·파라미터 동결·단발 평가 같은 되돌릴 수 없는 단계는 대표가 직접 승인한다.

쉽게 말하면

‘좋은 결과가 나오게 슬쩍 기준을 바꾸는 유혹’을 시스템으로 차단했습니다. 이건 단순한 칸
간함이 아니라, build-in-public 마케팅에서 ‘이 팀은 자기 결과를 부풀리지 않는다’는 신뢰
그 자체가 됩니다 — 경쟁사가 흉내 내기 가장 어려운 해자입니다.

9. 앞으로의 방향

9.1 기술 — 다음 마일스톤

- **실제 트레이스 적용(지금 직전 단계):** 본인 Claude Code 세션 로그나 공개 LangGraph 트레이스에 탐지기를 돌려 (a) 후보가 합리적으로 잡히는지, (b) 오탐이 합성처럼 0으로 유지되는지, (c) 실제 토큰 절감액이 얼마인지 측정.
- **비용 스토리의 핵심 = (c):** '토큰 낭비를 줄인다'는 주장은 합성 F1 이 아니라 실측 절감 숫자로만 뒷받침된다.
- **regen_handoff 복구:** 구조적으로 후보 경로가 없는 문제를 해결해 4 번째 패턴을 다시 범위에 포함.

9.2 제품 로드맵 (연구↔제품 플라이휠)

각 단계는 '앞 단계가 신호를 준 다음에만' 진행합니다. 한 신호에 사업 전체를 걸지 않습니다.

S0 낭비탐지(지금) → S1 실시간 차단 → S2 조정 관측(그래프) → S3 수렴/건강성 지능 → S4 Guardian(자율 감독) → S5 거버넌스

9.3 시장 진입(GTM) — 준비된 자산

이미 초안이 준비된 자산들입니다(별도 'Custos SNS' 운영 트랙에서 다룸).

- Build-in-public 스토리 시리즈(가설을 죽인 정직한 음성 결과 → 피벗 → 자기기만 순간 → 출시 직전 실사)
- Reddit 가치 우선 댓글(6 가지 반복 질문 유형) · Ask HN 글 · DEV.to 정직한 negative-result 아티클
- 한국 커뮤니티(Instruct.KR, 모두의연구소) · Discord · GitHub 이슈 작성자 콜드 DM 템플릿

10. 대표로서 꼭 기억할 것

회의·투자자 미팅·대외 글에서 흔들리지 않도록 정리한 핵심 체크리스트입니다.

주제	기억할 한 줄
우리가 파는 것	에이전트 '사이'의 토큰 낭비를 보이게 만드는 도구 (Clew by Custos)
가장 강한 증거	사전 약속한 단발 평가에서 GO — F1 0.857, 오탐 0 (합성 데이터)
가장 큰 한계	실제 트레이스 검증·실측 절감액·고객이 모두 아직 0
넘지 말 선	'프로덕션 절약' 단언 금지, IBM F1을 우리 성과로 말하지 않기, '4패턴 전부' 금지(3+1 디스코프)
진짜 해자	기술보다 '자기기만 방지' 규율 — 흥내 내기 가장 어려움
다음 한 걸음	실제 멀티에이전트 트레이스에서 오탐 0 유지 + 실측 절감액 확보
되돌릴 수 없는 결정	git 동결·파라미터 동결·단발 평가는 대표가 직접 최종 승인

마지막으로 — 이 프로젝트의 본질은 '똑똑한 탐지기'가 아니라 **'자기 결과를 부풀리지 않는 팀'**이라는 점입니다. 기술은 따라잡힐 수 있어도, 그 정직함이 만드는 신뢰는 가장 오래 가는 자산입니다.